

Encoding & Embedding

Encoding = numerical representation → vector → Count based method ← frequency
Embedding = meaningful numerical representation → vector → neural network or transformer

What is Encoding? ✓ text → zero

Encoding = converting raw data into numerical form (often sparse) without necessarily preserving semantic meaning

Count based, matrix factorization based

One-hot encoding, ✓

BOW, ✓ Bag of words

TF-IDF, = term freq, inverse document freq

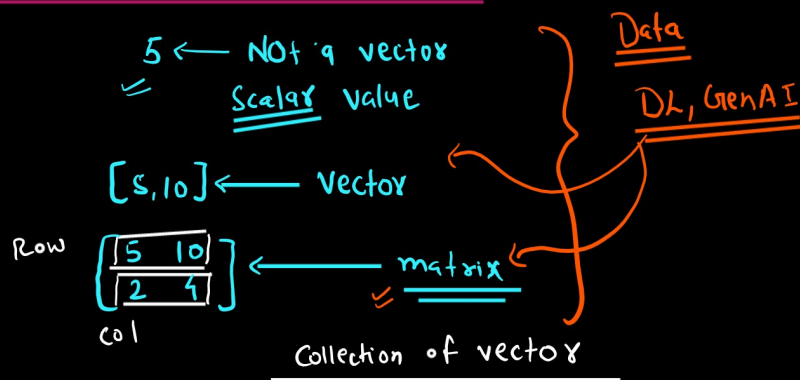
N-grams

BM25: BM25 is an improved extension of TF-IDF, but technically it is a ranking algorithm, not just an encoding method.

GloVe: GloVe (Global Vectors) is a count-based + matrix factorization-based

Practical
→ RAG → Retrieval

Vector $\rightarrow$ set of numerical value



Differences between scalar, vector and matrix

Scalar 5

Vector $[1, 2, 3]$

Matrix $\begin{bmatrix} 1, 2 \\ 3, 4 \end{bmatrix}$

Matrix = vectors ka collection

Vector = special case of matrix (1 row or 1 column)

Matrix = a table of numbers (grid)

A vector is a numerical representation of data in n-dimensional space.

In Mathematics Vector Comes from Linear Algebra

A vector = list of numbers $[1.2, -0.5, 3.8]$

In NLP = Embedding = vector with meaning

Capture semantic relationships

$[1, 2, 3, 4]$ 4-dim

$[1, 2, 3, 4, 5, 6]$
6dim

vector $\rightarrow [1, 2, 3, 4, 5, 6, 7 \dots n]$

mathematical magnitude + Direction

embedding $\rightarrow$ vector $\rightarrow$ vector of meaningful number

NLP(AI)

$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

This is called a 2×2 matrix.

2 rows

2 columns

More Examples

1. 3×2 Matrix

$B = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$

This is called a 3×2 matrix.

3 rows

2 columns

Row $\times$ Column
 $\uparrow \quad \uparrow$
 $\underline{3} \quad \underline{2}$

Vector $\leftarrow$ Collection of number

2. 2×3 Matrix

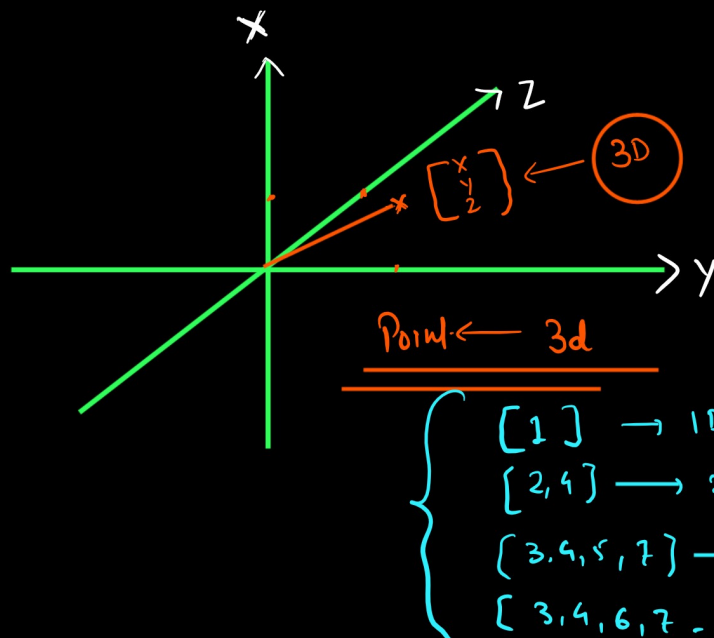
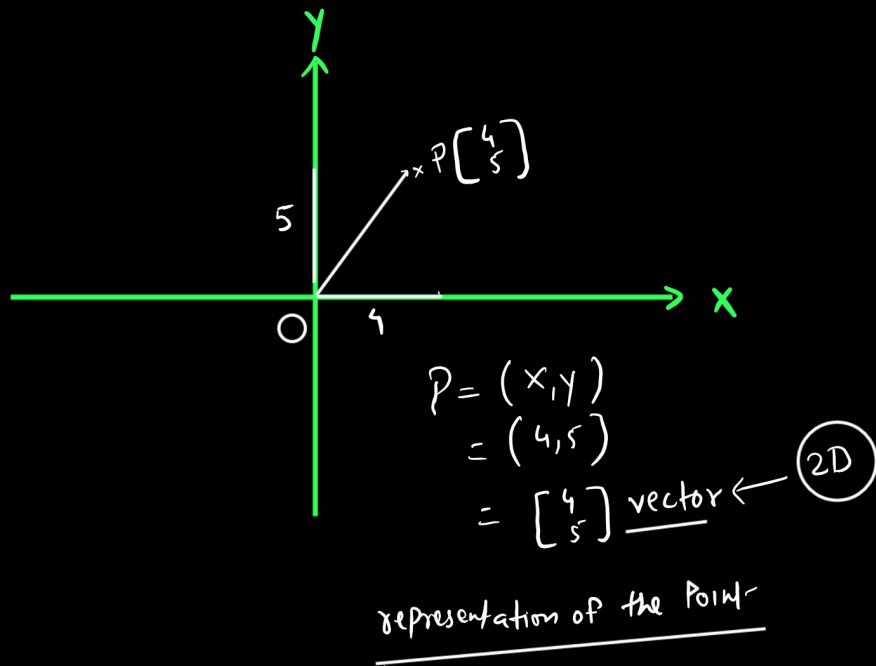
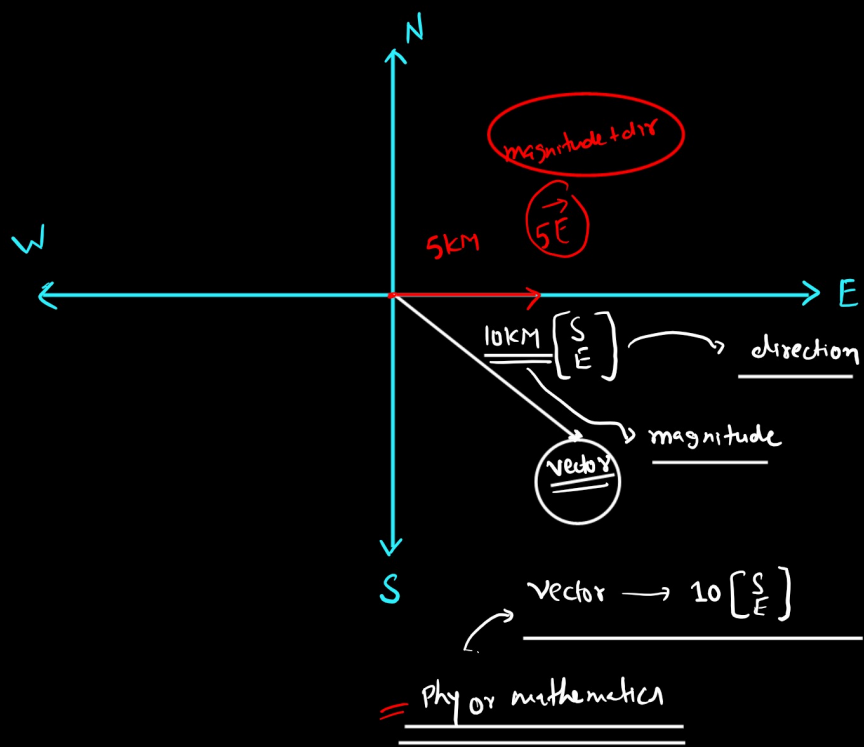
$C = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$

This is called a 2×3 matrix.

2 rows

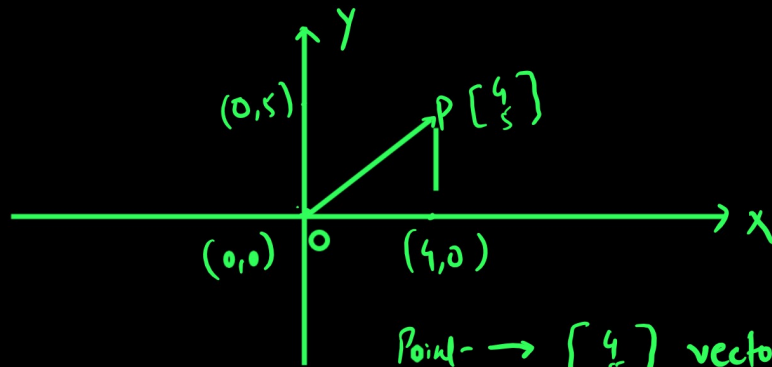
3 columns

Vector



OpenAI embedding model $\Rightarrow$ $\frac{3072}{\text{dim}}$

$\Rightarrow [1, 2, 3, 4, 5, \dots, 3072]$



Point $\rightarrow$ $\begin{bmatrix} 4 \\ 5 \end{bmatrix}$ vector representation

Similarity
Search

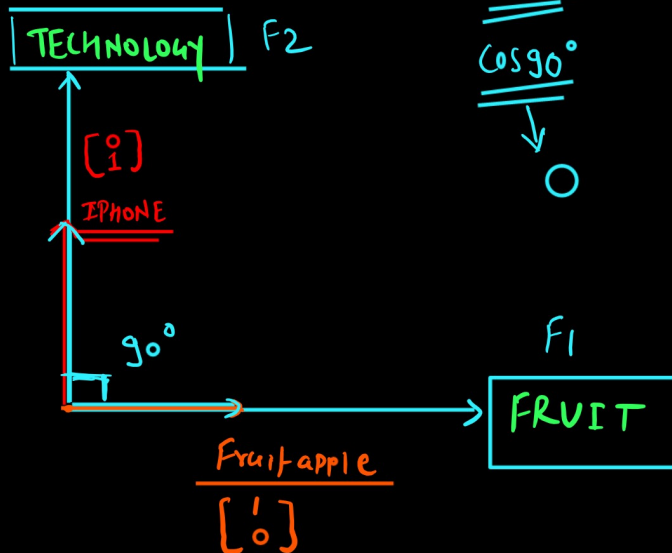
$$\begin{aligned} \vec{OP} &= \sqrt{(y_2 - y_1)^2 + (x_2 - x_1)^2} \\ &= \sqrt{(4 - 0)^2 + (5 - 0)^2} \\ \vec{OP} &= \sqrt{41} \end{aligned}$$

$$\vec{OP} = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$

Cosine
 $\cos 90^\circ$
 $\downarrow$
 $\circ$

1 IPHONE
2 APPLE } $\rightarrow$ Sim $\rightarrow$ $\circ$

Proof
Vector



IPHONE $\rightarrow [1, 0]$

Fruit apple $\rightarrow [0, 1]$

Similarity { (1) Dot Product $\begin{pmatrix} x_1 & y_1 \end{pmatrix} \begin{pmatrix} x_2 & y_2 \end{pmatrix}$ $\begin{matrix} x & y \\ [1, 0] \\ [0, 1] \end{matrix}$
 $(x_1 * x_2) + (y_1 * y_2)$
 $(1 * 0) + (0 * 1)$
 $= 0$

Encoding → Data → text → vector
 = [1, 2, 3, 4, 5, ..., n]

Count-
 { 1 ONE-HOT-ENCODING
 2 BOV
 3 TF-IDF }

Encoding

RANKING
BM-25
TF-IDF
GLOVE vector
 X

Embedding → data (text) → vector
 { Image
 video
 audio }
 [1, 2, 3, 4, 5, ..., n]

Neural network

transformer

SOTA

Corpus → Collection of documents

Document → Collection of paragraphs

Paragraph → Collection of sentences

Sentence → Collection of words/tokens

Word (Token) → May split into tokens

Character → Smallest unit

I LOVE indig
 word → token

A, B, C, D, E, ..., Z

Embedding → early → 2018-19-20

word → embedding
 < word 2 vec >
 < FASTTEXT >
Neural network

State of art
late

SOTA

transformer

word → embd

Sent. → " "

Doc → " "

... → " "

Image → embedding

audio → " "

video → " "

What is an Embedding?

Embedding is mathematical representation of the data

Embedding is a way to convert text, image, audio, video into meaningful numbers.

Embedding is dense, high-dimensional numerical vector that represents the semantic meaning of data.

Dense: Non Spars

High-dimensional: "Typically ranges from 100 to several thousand dimensions depending on model"

384, 768, 1024, 1536, 2048 etc....

vector : Mathematical object in linear algebra In programming this vector is typically stored as an array, means a collection of numerical values(float).

Converts data into a numerical vector that preserves meaning .

Embedding("king") Embedding("queen") Distance between them → small Angle will be less between two vector

Embedding("king") Embedding("banana") angle between huge Distance → large

Classical Word Embedding Models

- Word2Vec , FastText

Limitation:

- They create embeddings at the word level only.
- They produce static embeddings (fixed vectors).
- They do not understand context.

Example

Sentence 1: I sat on the river bank.

Sentence 2: I deposited money in the bank.

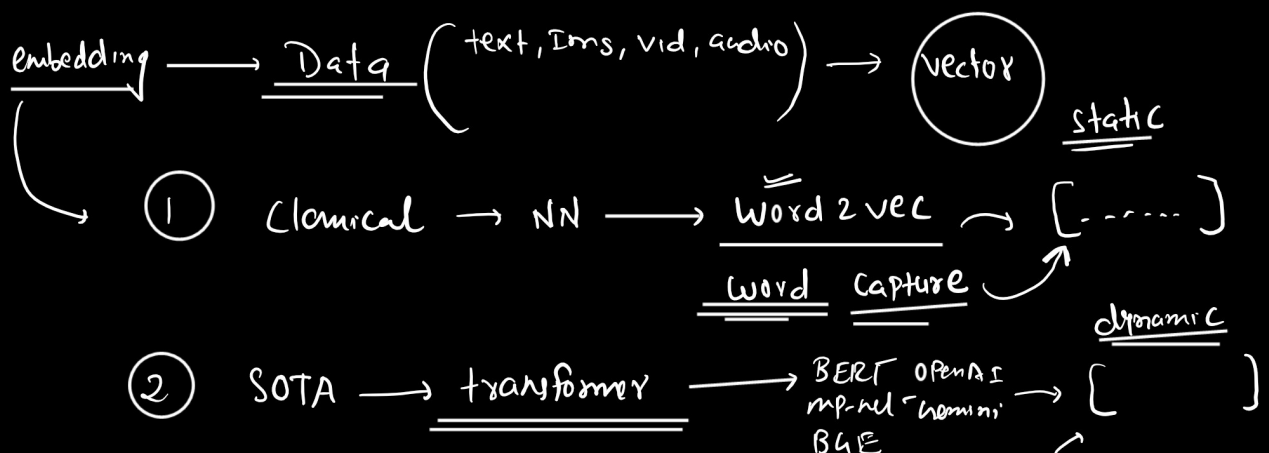
Word: bank

In a static embedding model (word2vec), the word "bank" is represented by the same vector in both sentences.
bank → [0.21, -0.44, 0.89, ...]

In SOTA Transformer-based Embeddings Model , the word "bank" will be represented by the different vector.

bank → [0.21, -0.44, 0.89, ...]

bank → [0.93, 0.01, 0.75, ...]

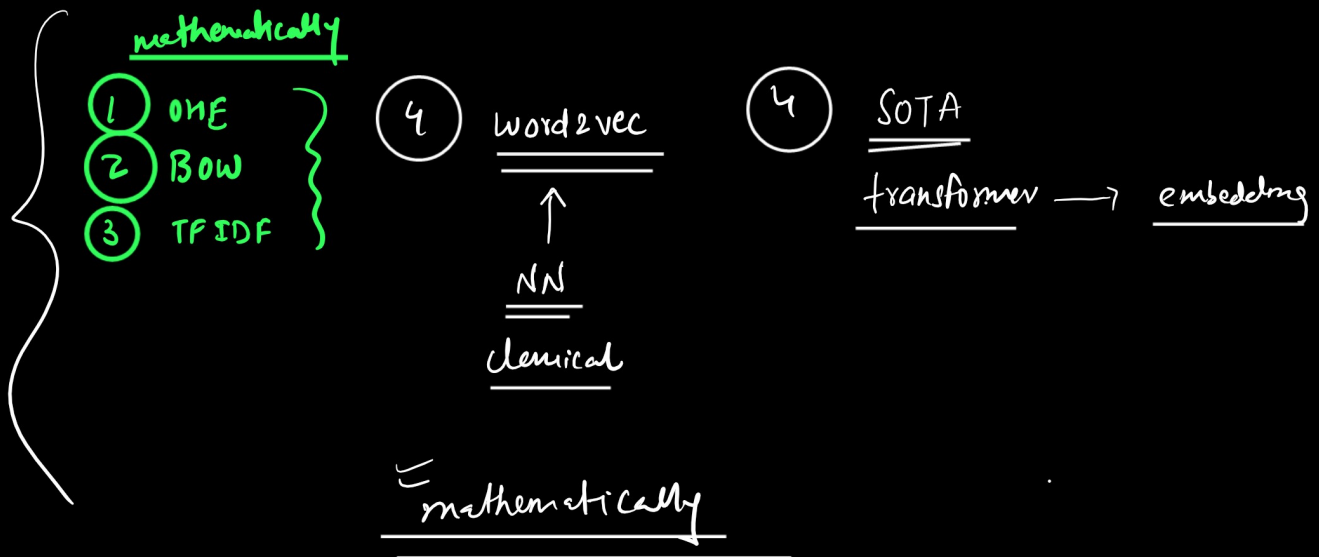


What is a difference between encoding and embedding?

Feature	Encoding(frequency)	Embedding
Purpose	Convert data to numbers	Convert data to numbers (Represent meaning)
Nature	Often sparse(count based)	Dense(Neural Network or Transformer based)
Semantic meaning	Not preserved	Preserved
Context awareness	Not preserved	Preserved

Model Name	Modality	Provider	Category
text-embedding-3-small	Text	OpenAI	Closed
text-embedding-3-large	Text	OpenAI	Closed
text-embedding-ada-002 (legacy)	Text	OpenAI	Closed
Gemini Embeddings	Text	Google	Closed
Claude Embeddings	Text	Anthropic	Closed
Cohere embeddings	Text	Cohere	Closed
Titan Text Embeddings	Text	AWS (Amazon Bedrock)	Closed

all-mpnet-base-v2	Text	HuggingFace (Sentence-Transformers)	Open
all-distilroberta-v1	Text	HuggingFace (Sentence-Transformers)	Open
all-roberta-large-v1	Text	HuggingFace (Sentence-Transformers)	Open
msmarco-miniLM-v3	Text	HuggingFace (Sentence-Transformers)	Open
BAAI-BGE-Base-En	Text	HuggingFace (BAAI)	Open
BAAI-BGE-Large	Text	HuggingFace (BAAI)	Open
BGE-multilingual	Text	HuggingFace (BAAI)	Open
e5-small	Text	HuggingFace	Open
e5-base	Text	HuggingFace	Open
e5-large	Text	HuggingFace	Open
st5-xxl-embed	Text	HuggingFace	Open
miniLM L6	Text	Microsoft(OpenAI)	Open
miniLM L12	Text	Microsoft(OpenAI)	Open
OpenCLIP	Image (Vision)	HuggingFace (OpenAI)	Open
BLIP2 embedding models	Image (Vision)	Various	Open
DINO-v2 embeddings	Image (Vision)	Meta	Open
Wav2Vec 2.0	Audio	Meta	Open
HuBERT	Audio	Meta	Open
YAMNet	Audio	Google	Open
CLAP (Contrastive Language-Audio Pretraining)	Audio	LAION	Open
VideoMAE	Video	Meta	Open
i3D (Inflated 3D ConvNet)	Video	DeepMind	Open



What is a semantic search (dot product and cosine similarity)

Semantic search = Meaning-based intelligent search using embeddings.

meaning (transformer)
[] $\leftarrow$ vector

Semantic search means searching based on meaning, not just exact words.

Semantic search is usually performed using:

Cosine similarity

Dot product

Euclidean

Example:

• eating fiber reduces heart risk

• eating fruits and vegetables lowers cardiovascular disease chances

} Similar

Both sentences have the same meaning

Even though the words are different, embeddings place them close together in vector space.

That's why semantic search returns relevant results even when wording changes.

Example:

• Eating fiber reduces heart risk

• Buying a new car improves driving comfort

~~Similar~~

Dissimilar

Vector

vector

Completely different topics

Health vs Automobile

Embeddings will be far apart

Low cosine similarity

Data $\rightarrow$ embedding vector $\rightarrow$ [.....]

Context

What other similar vector?

Dot Product, ED, CS

Semantic search
Similarity search

[] $D_2 \leftarrow Data$
[] $D_3 \leftarrow Data$
[] $D_4 \leftarrow Data$

What is a keyword search vs semantic search(similarity search)

Keyword vs Semantic Search(Similarity Search)

Clean Comparison Line

Keyword search matches words.

Semantic search matches meaning: find out similarity using matrix like cosine, Euclidian, dot product etc.

How Keyword-Based Search Work

If you search:

"How to reduce heart risk"

≡ Sent

The system will mainly look for sentence containing those exact words like:

• reduce

• heart

• risk

If a sentence instead contains:

"Ways to lower cardiovascular disease chances" Sent

semantically
same meaning
similar

A traditional keyword search may not match it because:

• reduce ≠ lower

• heart risk ≠ cardiovascular disease

Even though the meaning is similar, the words are different

Example

Sen1: how to lose weight fast

Sen2: quick fat loss strategies

making sense

Keyword

Keyword search → weak match

Semantic search → strong match

vector
semantic → meaning
transformer

Application of Embedding

1. Semantic Search (Most Fundamental): Meaning-based search

Instead of matching keywords, we match vector similarity.

2. Google-like search engines

3. Google Image search

4. Pinterest

User Search: "a red car" System retrieves: "Images of red cars"

5. Recommendation Systems

Netflix: Similar movies

Amazon: Similar products

Spotify: Similar songs

All use vector similarity.

6. Topic Modeling (Clustering)

7. Group similar documents together

News categorization

8. RAG (Retrieval-Augmented Generation)

embedding

Data → Neural vector

Retrieval
Semantic Search

Similarity

① Encoding

ONE
BOW
TFIDF

mathematical
30 min/-

NN

mathematically

② Embedding

→ derived → word → word2vec

SOTA → word
text (Sent, Document, book)
img
vid
audio → transformer

① vector

② Dot Prod
cosine simi
ED

③ Encoding & Embedding

④ Semantic Search (Similarity Search)

⑤ keyword search vs SS (Semantic Search)

⑥ Application